A Security Research Publication by Yonas Abeselom

THE UNAUDITED DRIVE

Why No One Has Verified That Your NVMe Sanitize Firmware Actually Works

Yonas Abeselom

Independent Security Researcher | github.com/yonasabeselom/aad50

June 2026

Contents

Chapter One The Question Nobody Is Asking

Chapter Two How Sanitize Is Supposed to Work

Chapter Three What Self-Certification Actually Means

Chapter Four The Only Independent Audit That Exists

Chapter Five What They Found

Chapter Six What Has Changed Since 2011

Chapter Seven What Operator-Level Verification Looks Like

Chapter Eight AAD-50 — The Open Source Response

Chapter Nine What You Should Do

The Researchers Behind This Story

This book draws on two landmark peer-reviewed studies. Here is who conducted them and why their work matters.

Table 0. Key Researchers Referenced in This Book

Researcher	Role at Time	Now	Contribution
Michael Wei	PhD Student, UC San Diego	Senior Research Scientist, VMware AI Labs	Lead author of FAST 2011. Built 'Ming the Merciless' — custom FPGA hardware to read raw NAND data bypassing the FTL. Designed all recovery experiments.
Laura M. Grupp	PhD Student, UC San Diego	Storage Systems Researcher, Microsoft Azure	Co-author FAST 2011. Flash memory characterisation and error rate analysis across drive manufacturers.
Frederick E. Spada	Researcher, UC San Diego CMRR	—	Co-author FAST 2011. Hardware specialist for chip-level extraction and FPGA interface design.
Steven Swanson	Professor, UC San Diego	Director, Non-Volatile Systems Lab, UCSD	Senior author and faculty supervisor. One of the most cited researchers in flash storage systems. Also authored the SAFE follow-up proposal (2010).
Md Mehedi Hasan	PhD Student, Univ. of Alabama Huntsville	Security Researcher	Lead author of USENIX Security 2020 paper. Demonstrated analog threshold voltage recovery from scrubbed NAND using standard digital interfaces.
Biswajit Ray	Professor, Univ. of Alabama Huntsville	Associate Professor, UAH	Senior author of USENIX Security 2020. Research focus: flash memory reliability and security at the analog level.

Wei et al. (2011) refers to the four-person team above from UC San Diego. When this book cites 'Wei et al. USENIX FAST 2011' it means their paper: 'Reliably Erasing Data from Flash-Based Solid State Drives,' published at the 9th USENIX Conference on File and Storage Technologies, San Jose, California, 2011, pages 105-117. Freely available at: usenix.org/legacy/events/fast11/tech/full_papers/Wei.pdf

Hasan and Ray (2020) refers to the two-person team above from the University of Alabama in Huntsville. When this book cites 'Hasan and Ray USENIX Security 2020' it means their paper: 'Data Recovery from Scrubbed NAND Flash Storage: Need for Analog Sanitization,' published at the 29th USENIX Security Symposium, 2020. Freely available at: usenix.org/conference/usenixsecurity20/presentation/hasan

CHAPTER One

The Question Nobody Is Asking

Every year, millions of NVMe solid-state drives are decommissioned. They leave data centres, hospitals, law offices, government buildings, and corporate IT departments. Before they go, someone runs a sanitize command. The command returns success. The drive is cleared. Job done.

But here is the question that almost nobody asks: how do you know the drive actually did what it said it did?

Not how do you know in theory, based on the manufacturer's documentation. How do you know in practice, on this specific drive, on this specific firmware version, today?

"One drive reported SUCCESS while all data remained completely intact. The drive lied."

The answer, for most organisations running most tools on most drives, is: you do not know. You trust. You assume. You move on.

This book is about why that trust is not always warranted — and what you can do about it.

CHAPTER Two

How Sanitize Is Supposed to Work

Modern NVMe drives implement a command called Sanitize — Opcode 0x84 in the NVMe Base Specification. When issued, this command instructs the drive's internal controller to destroy all data on the device, including regions invisible to the host operating system: over-provisioned zones, bad block retirement pools, and wear-levelling reserves.

This is fundamentally different from software overwrite tools. Tools like DBAN, nwipe, and DoD 5220.22-M push data from the host CPU across the PCIe bus, through the operating system storage stack, and into the drive as normal write commands. The drive's Flash Translation Layer (FTL) intercepts these writes and redirects them to fresh physical blocks, leaving the original data in hidden regions the host can never reach.

> **The NVMe Sanitize command bypasses this entirely. The host sends one small command packet. The drive's ASIC executes the destruction internally at silicon speeds on its own internal buses. No data travels across PCIe during the actual sanitization.**

The sanitize command supports three actions, each targeting a different destruction vector:

Table 1. NVMe Sanitize Action Codes (Opcode 0x84)

Action	CDW10 Code	What It Does
Overwrite	0x02	Physically overwrites all NAND cells with a specified pattern
Block Erase	0x01	Erases all blocks including FTL mapping tables
Crypto Erase	0x04	Destroys the internal Media Encryption Key registers

In principle, a correctly implemented NVMe Sanitize command is the right tool for the job. In practice, the word 'correctly' carries significant weight.

CHAPTER Three

What Self-Certification Actually Means

When a drive manufacturer ships a product claiming NVMe Sanitize support, they are self-certifying. There is no mandatory independent third-party audit of sanitize firmware correctness required by any current standard — including NIST SP 800-88 Rev. 2, IEEE 2883-2022, or the NVMe Base Specification itself.

The certification process works roughly like this: the manufacturer implements the sanitize command in firmware, runs internal tests, reports supported capabilities in the SANICAP field of the Identify Controller response, and ships the product. A compliance officer at a purchasing organisation reads that SANICAP field, notes that sanitize is supported, and proceeds with procurement.

> *"There is no mandatory independent audit of sanitize firmware correctness required by any current standard."*

Nobody in this chain independently verifies that the sanitize command does what the manufacturer claims. The buyer trusts the seller. The compliance officer trusts the datasheet. The IT administrator trusts the command return code.

This would be acceptable if drive firmware were reliably correct. As the next chapter shows, it is not.

CHAPTER Four

The Only Independent Audit That Exists

In 2011, researchers Michael Wei, Laura Grupp, Frederick Spada, and Steven Swanson at the University of California San Diego published 'Reliably Erasing Data from Flash-Based Solid State Drives' at the USENIX Conference on File and Storage Technologies (FAST '11). It remains the most comprehensive independent evaluation of SSD sanitize firmware ever conducted.

Their methodology was straightforward: acquire real drives, write known data to them, run sanitization procedures, then attempt recovery using every available technique — including direct chip extraction and custom FPGA-based hardware designed specifically to bypass the FTL and read raw NAND data.

> **They tested 15 widely-used sanitization protocols on 12 real ATA/SCSI solid-state drives — including DoD 5220.22-M, Gutmann 35-pass, German VSITR, British HMG IS5, US Air Force 5020, and Mac OS X Secure Erase. Every single one failed.**

The failure on software overwrite tools was expected and understood — the FTL intercepts host writes and stale data persists in hidden regions. What was more alarming was what they found when they evaluated the drives' own built-in sanitize commands.

CHAPTER Five

What They Found

Wei et al. evaluated 12 drives with built-in sanitize commands — the ATA SECURITY ERASE UNIT and ACS-2 SANITIZE BLOCK ERASE commands that were the precursors to today's NVMe Opcode 0x84. Their findings were sobering.

Table 2. Wei et al. Built-in Sanitize Command Results (FAST 2011)

Finding	Detail
Drives tested with built-in sanitize	12 drives across multiple manufacturers
Drives that executed correctly	4 of 12 — only 33%
Drives that silently failed	3 of 12 — including one that reported SUCCESS with all data intact
Drives requiring specific conditions	2 of 12 — only worked under firmware reset
Overall failure rate	Approximately 67% failed to sanitize correctly

> *"Eight of the drives reported they supported ATA SECURITY... only four executed the ERASE UNIT command reliably." — Wei et al., USENIX FAST 2011*

The most alarming finding was a drive that reported the sanitize command completed successfully — returning a success status code to the host — while the underlying NAND cells had not been touched. The drive lied. And without independent verification of completion, the operator had no way to know.

Wei et al.'s conclusion was unambiguous: reliable SSD sanitization requires built-in, verifiable sanitize operations — not just operations that claim to verify themselves.

CHAPTER Six

What Has Changed Since 2011

The honest answer is: very little, in the ways that matter most.

The interface has changed. ATA and SCSI have largely been replaced by NVMe in enterprise and consumer storage. The NVMe Sanitize command (Opcode 0x84) is more capable than its ATA predecessors — it uses NSID=0xFFFFFFFF to broadcast to all physical blocks including over-provisioned regions, directly addressing the FTL coverage problem Wei documented.

But the fundamental problem Wei identified — firmware that reports success without executing correctly — has not been solved by the interface change. NVMe drive firmware can contain the same class of bugs. And the tooling situation has remained problematic.

> **The current nvme sanitize command in nvme-cli — the standard Linux NVMe tool — dispatches the sanitize command and returns to the user immediately after command acceptance. It does not poll Log Page 0x81 (Sanitize Status) to confirm the drive actually completed the background operation. Most operators never separately verify completion.**

This was confirmed in June 2026 by a nvme-cli contributor on RFC #3415: 'The sanitize command does not poll the log page but just only submit the command and return the status if any error caused.'

In other words: the tooling assumes the drive completed the operation correctly. If the firmware silently fails — exactly as Wei documented in 2011 — the operator has no way to know. The situation Wei described fifteen years ago remains largely unchanged at the tooling level.

CHAPTER Seven

What Operator-Level Verification Looks Like

If manufacturers do not independently audit their firmware, and if standard tooling does not verify completion, the question becomes: what can an operator do to verify that a sanitize operation actually completed correctly?

The NVMe specification provides the answer: Log Page 0x81, also known as the Sanitize Status log page. After a sanitize command is dispatched, the host can poll this log page and read the SSTAT field to determine the actual hardware completion status.

Table 3. NVMe Log Page 0x81 SSTAT Codes and Correct Responses

SSTAT Code	Meaning	Correct Response
0x0	Idle — no recent sanitize	Treat as complete
0x1	Completed successfully	Confirmed — advance
0x2	Sanitize in progress	Continue polling (2 second interval)
0x3	Completed with errors	Abort — record fault in audit log

Polling Log Page 0x81 after every sanitize cycle — and refusing to advance until SSTAT = 0x1 confirms hardware completion — is the operator-level equivalent of the independent audit that manufacturers do not provide. It does not trust the drive's self-reported completion. It independently confirms it.

Following RFC #3415, a nvme-cli contributor opened PR #3438 implementing a --wait flag for nvme sanitize that addresses the fire-and-forget behaviour at the basic tooling level. This is a significant step forward. But confirmation of a single cycle is only the beginning of a complete high-assurance sanitization workflow.

CHAPTER Eight

AAD-50 — The Open Source Response

The Abeselom ASIC-Direct 50 (AAD-50) is a firmware-enforced, 50-cycle NVMe sanitization protocol designed to address the verification gap Wei et al. identified and to provide a complete, auditable data destruction workflow for compliance contexts.

AAD-50 does not trust drive firmware. It verifies it — after every single cycle, using Log Page 0x81 SSTAT polling. A drive that silently fails a sanitize cycle is detected immediately and the operation is aborted with a fault record in the audit log.

Table 4. AAD-50 Three-Phase Destruction Matrix

Phase	Cycles	Action	Verification
B — Physical Overwrite	1–40	NAND cell overwrite (CDW10=0x02)	Log Page 0x81 per cycle
C — FTL Teardown	41–45	FTL index destruction (CDW10=0x01)	Log Page 0x81 per cycle
A — Crypto Seal	46–50	MEK key destruction (CDW10=0x04)	Log Page 0x81 per cycle

The 40-cycle Phase B allocation is primarily justified by firmware fault redundancy — the same logic that makes RAID arrays use multiple disks: if one cycle fails silently despite the polling safeguard, overlapping cycles provide redundancy. Multiple cycles with per-cycle hardware confirmation is the correct engineering response to a 25% documented firmware failure rate.

> **AAD-50 produces a SHA-256 tamper-evident audit chain across all 50 cycles and a PDF Certificate of Destruction with operator name, drive serial number, and compliance alignment — the documentation compliance officers and auditors need.**

AAD-50 is free and open source. Windows GUI, Linux CLI, and standalone EXE are all available. The full specification, IEEE-format whitepaper, and reference implementation are at:

https://github.com/yonasabeselom/aad50

CHAPTER Nine

What You Should Do

The practical steps depend on your context. Here is a clear framework based on threat model and compliance requirements.

Table 5. Recommended Sanitization Approach by Context

Context	Minimum Recommendation	High Assurance
Personal laptop resale	Single nvme sanitize --wait	AAD-50 dry-run then live
Small business drive retirement	nvme sanitize --wait + verify log	AAD-50 with PDF certificate
HIPAA / GDPR compliance	AAD-50 — PDF certificate required	AAD-50 + retain audit records
Enterprise ITAD	AAD-50 — auditable chain of custody	AAD-50 + third party verification
Defence / intelligence	AAD-50 + physical destruction	Physical destruction always

For any context where you need to prove — not just assume — that a drive is forensically clean, the minimum requirement is hardware-confirmed completion via Log Page 0x81. The --wait flag in nvme-cli (PR #3438) provides this at the basic level. AAD-50 provides it across 50 cycles with a full compliance documentation package.

"The question is not whether your sanitize command worked in theory.
The question is whether you can prove it worked in practice."

The answer to the original question — has anyone audited NVMe sanitize firmware? — remains no. Until that changes, operator-level verification is the only available defence. AAD-50 provides it, free, open source, for every NVMe drive on Linux and Windows.

About the Author

Yonas Abeselom is an independent security researcher based in Addis Ababa, Ethiopia. He holds a BSc in Computer Science and a Diploma in Information Technology. AAD-50 is his open-source contribution to the NVMe sanitization problem that has remained unsolved in the public domain since Wei et al. identified it in 2011.

Following publication of AAD-50, Peter Gutmann engaged with the RFC, Bruce Schneier replied the same day, NVM Express initiated internal review, and nvme-cli contributor ikegami-t opened PR #3438 implementing the --wait flag based on RFC #3415 — all within 10 days of publication.

Contact: yonas_abeselom@protonmail.com

Repository: https://github.com/yonasabeselom/aad50

RFC #3415: https://github.com/linux-nvme/nvme-cli/issues/3415